UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

APUNTES EXTENDIDOS

Sistemas Distribuidos

Clase 06 – Sistemas de almacenamiento

Departamento de Computación

Ingeniería Informática – Comisión A

Pilar – Segundo cuatrimestre 2026

Equipo docente

2026

1 Propósito de la clase

Las primeras cinco clases construyeron un lenguaje para razonar sobre incertidumbre, comunicación remota, replicación, consenso y transacciones. La clase 6 cambia deliberadamente de nivel: ya no pregunta solamente *qué garantía queremos*, sino *qué debe ocurrir físicamente para que esa garantía siga siendo verdadera después de una falla*.

Cuando una API devuelve `200 OK`, cuando una réplica declara una entrada *committed* o cuando una base de datos confirma `COMMIT`, existe una cadena de mecanismos que transforma una intención lógica en bytes recuperables. En esa cadena aparecen buffers de proceso, page cache del sistema operativo, registros de log, sincronización con el dispositivo, páginas de datos, versiones, índices, checkpoints y tareas de mantenimiento.

> El almacenamiento no es una caja opaca situada debajo del sistema distribuido. Es el lugar donde las promesas lógicas se convierten en órdenes de escritura, esperas, recuperación y riesgos medibles.

Al finalizar esta clase, el estudiante debería poder:

- relacionar timeout, retry, idempotencia, replicación, consenso y transacciones con persistencia;
- distinguir una operación aceptada, escrita, replicada, aplicada y durable;
- explicar la regla de Write-Ahead Logging y construir un timeline de crash recovery;
- explicar qué problema resuelve un checkpoint y qué costo introduce;
- razonar sobre visibilidad de versiones, snapshots y garbage collection en MVCC;
- leer un plan simple de `EXPLAIN` y justificar un índice mediante evidencia;
- explicar el flujo de escritura y lectura de una LSM-tree;
- interpretar write, read y space amplification, backlog de compaction y write stalls;
- comparar B-tree y LSM-tree desde la carga, la garantía y las métricas, no desde preferencias de producto.

2 Repaso recuperativo de las clases 1 a 5

El repaso no debe entenderse como una repetición abreviada de las clases anteriores. Su propósito es recuperar una sola conexión por clase y mostrar por qué el storage engine se vuelve necesario.

2.1 Clase 1: incertidumbre y observación parcial

En un sistema distribuido, un cliente no observa directamente el estado interno del servidor. Si una respuesta no llega, no puede distinguir con certeza entre request perdido, servidor lento, servidor caído, operación ejecutada con respuesta perdida o partición de red.

La conexión con almacenamiento es directa: si el cliente puede reintentar una operación cuya ejecución desconoce, el servidor necesita conservar evidencia durable de la intención lógica y del resultado producido.

> La incertidumbre del cliente exige memoria durable del lado servidor.

2.2 Clase 2: RPC, timeout, retry e idempotencia

Un retry mejora progreso frente a fallas transitorias, pero también puede repetir un efecto ya realizado. Para una operación no naturalmente idempotente, como cobrar una compra, el servidor suele recibir una clave estable y tratar varios intentos físicos como una sola operación lógica.

Una implementación razonable debe persistir, como una unidad coherente:

1. la clave idempotente;
2. el estado de la operación;
3. el efecto realizado;
4. la respuesta que debe devolverse ante nuevos intentos.

Si la API responde antes de que esa relación sea durable, un crash puede borrar la evidencia y convertir el siguiente retry en un segundo cobro.

> La idempotencia de una API depende también del contrato de durabilidad del almacenamiento.

2.3 Clase 3: replicación y consistencia

Replicar y persistir no son sinónimos. Tres copias en RAM pueden perderse ante una falla común; una copia durable en un único dispositivo puede sobrevivir al crash del proceso, pero no a la pérdida del dispositivo. Además, varias copias durables pueden representar versiones distintas.

Por eso una política de confirmación debe declarar qué evento considera suficiente:

- recepción en memoria local;
- append a un buffer de WAL;
- escritura al kernel;
- flush durable local;
- persistencia en una réplica remota;
- persistencia en una mayoría;
- aplicación a la estructura de datos principal.

Cada opción cambia latencia, disponibilidad, RPO y el conjunto de fallas toleradas.

> Más copias no determinan por sí solas ni durabilidad ni consistencia.

2.4 Clase 4: consenso y orden común

Raft o Paxos pueden acordar qué entrada pertenece al log lógico. Sin embargo, sus demostraciones de seguridad suponen que ciertos estados sobreviven a los reinicios. Un nodo que olvida un voto o un término puede responder de manera incompatible con su historia anterior.

> El consenso decide qué debe recordarse; el almacenamiento determina si realmente puede recordarse después de un crash.

Esto explica por qué los protocolos de consenso especifican qué datos deben persistirse antes de emitir determinadas respuestas.

2.5 Clase 5: transacciones y serializabilidad

Las transacciones definen un contrato lógico. Atomicidad exige no exponer medias transacciones; durabilidad exige que un commit sobreviva; aislamiento define qué historias concurrentes pueden observarse; consistencia exige preservar invariantes.

WAL, recovery, locks, validación, MVCC e índices son mecanismos que participan en la materialización física de ese contrato. Sin embargo, ningún mecanismo aislado equivale a ACID completo.

> La clase 5 definió el contrato; la clase 6 estudia cómo parte de ese contrato se sostiene sobre almacenamiento real.

3 Cómo estudiar este capítulo

Este capítulo está organizado en tres pasadas. La primera construye una intuición con un caso real; la segunda formaliza el mecanismo y sus supuestos; la tercera somete la solución a una falla o a una medición. Para estudiar de manera eficaz:

1. explique primero el problema sin mencionar una tecnología;
2. dibuje la ejecución normal y mueva el punto de falla;
3. identifique la garantía exacta y su frontera;
4. anote qué costo se desplaza hacia latencia, almacenamiento, reparación u operación;
5. cierre proponiendo una métrica o un experimento que podría refutar su diseño.

Las analogías del capítulo son puertas de entrada, no definiciones finales. Una respuesta completa debe poder abandonar la analogía y utilizar el vocabulario técnico correspondiente.

4 Caso conductor: una compra confirmada en el campus

Consideremos una aplicación de compra para el comedor universitario. Una estudiante confirma un pedido y recibe `200 OK`. Un segundo después, el servidor pierde energía.

El sistema debe preservar tres propiedades:

1. al reiniciar, el pedido confirmado debe existir;
2. un retry del cliente no debe generar un segundo cobro;
3. un reporte largo de ventas debe observar un estado coherente mientras siguen entrando compras.

Estas tres propiedades conducen a tres preguntas diferentes:

- **Durabilidad:** ¿qué debe cruzar una frontera física antes de responder?
- **Visibilidad:** ¿qué versión ve cada transacción concurrente?
- **Acceso:** ¿cómo encuentra el motor las filas requeridas sin recorrer todo el conjunto?

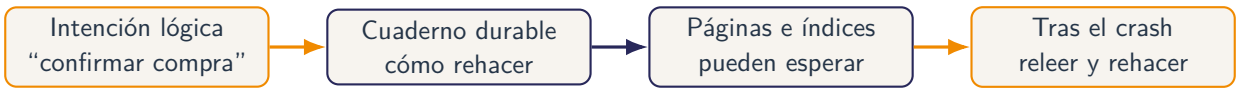
El caso aparecerá de nuevo en WAL, MVCC e índices. Más adelante se contrastará con una carga diferente: telemetría de aulas, energía y ocupación del campus.

5 Cinco intuiciones antes del detalle

Antes de formalizar, conviene construir una imagen mental de cada mecanismo. Las analogías siguientes no sustituyen la definición técnica: sirven para responder primero *qué problema intenta resolver* cada concepto. Después, cada intuición será sometida a reglas, fallas y métricas.

5.1 WAL: el cuaderno de evidencia

Imagine un libro de cuentas cuyas páginas son costosas de reescribir. Antes de modificar esas páginas, el motor anota en un cuaderno durable qué cambio debe poder reconstruir. Ese cuaderno es el *write-ahead log*.



- **Qué es:** una secuencia de registros que describe cambios y decisiones de commit.
- **Por qué existe:** persistir secuencialmente una historia compacta suele ser más barato que forzar cada página modificada en el camino crítico.
- **Qué permite:** recuperar cambios confirmados cuyas páginas todavía estaban atrasadas.
- **Qué no resuelve solo:** idempotencia de negocio, replicación, backups o consenso global.

La frase intuitiva que debe quedar es:

> Primero persisto la evidencia que me permite reconstruir; después puedo materializar las páginas con más libertad.

5.2 MVCC: conservar ediciones

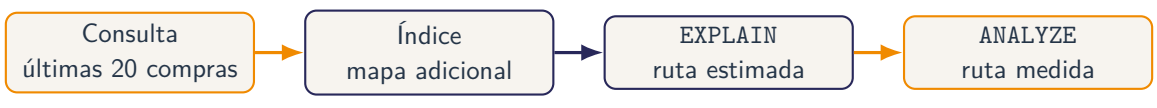
Un documento compartido no necesita borrar una edición apenas aparece otra. Puede conservar ambas y mostrar a cada lector la edición coherente con el momento en que comenzó. MVCC aplica esa idea a filas o claves.



- **Qué es:** control de concurrencia mediante múltiples versiones y reglas de visibilidad.
- **Por qué existe:** permite que lectores y escritores progresen con menos bloqueo mutuo.
- **Costo desplazado:** historia retenida, garbage collection, bloat y metadatos.
- **Límite:** MVCC es un mecanismo; el nivel de aislamiento determina qué anomalías siguen permitidas.

5.3 Índice y `EXPLAIN`: mapa y navegador

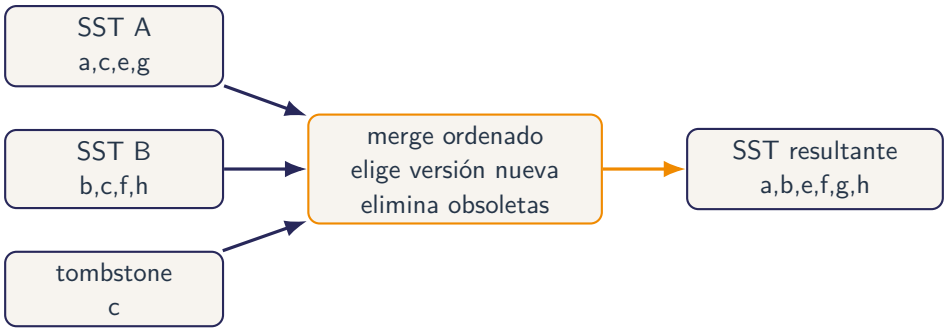
Un índice se parece a un mapa adicional que reduce el espacio de búsqueda. `EXPLAIN` se parece al navegador que muestra la ruta que el optimizador planea tomar; `EXPLAIN ANALYZE` recorre esa ruta y agrega observaciones reales.



- **Qué es un índice:** una estructura redundante que organiza claves para responder ciertas preguntas con menos trabajo.
- **Qué es `EXPLAIN`:** una representación del plan elegido por el optimizador y de sus estimaciones.
- **Qué agrega `ANALYZE`:** ejecuta la consulta y reporta filas y tiempos observados; con `BUFFERS`, agrega evidencia de I/O lógico.
- **Trade-off:** cada índice que acelera algunas lecturas encarece inserciones, actualizaciones, WAL, espacio y mantenimiento.

5.4 Compaction: ordenar las pilas pendientes

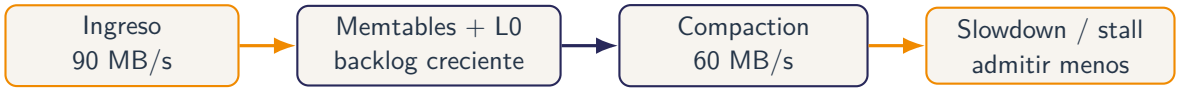
Una LSM-tree acepta escrituras rápidas formando archivos ordenados e inmutables. Esos archivos se acumulan como pilas. Compaction fusiona pilas, conserva la versión vigente y elimina información obsoleta cuando es seguro.



Compaction no es una tarea cosmética. Consume lectura, escritura, CPU y espacio temporal. Si su capacidad sostenida es menor que el ingreso, la deuda física crece.

5.5 Write stall: el molinete de estabilidad

Cuando llegan datos más rápido de lo que flush y compaction pueden absorber, el motor no puede crear capacidad por configuración. Solo puede acumular temporalmente, desacelerar, rechazar o redistribuir carga. Un *write stall* es una forma de backpressure: reduce admisión para impedir una deuda sin límite.



La lectura correcta es doble:

- un stall puntual puede ser un mecanismo sano de autoprotección;
- stalls frecuentes o prolongados son evidencia de que capacidad, configuración o patrón de carga no están alineados.

5.6 Mapa de preguntas

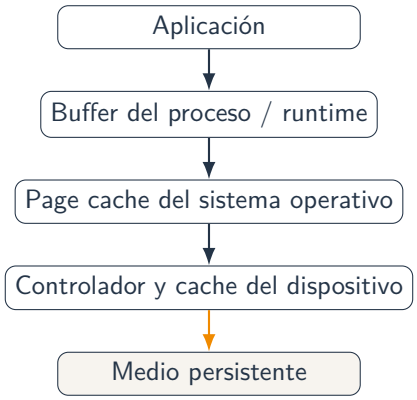
Concepto	Pregunta intuitiva	Evidencia que lo vuelve técnico
WAL	¿Cómo sobrevive el commit?	orden de registros, flush, recovery, RPO/RTO
MVCC	¿Cómo leen mientras otros escriben?	snapshots, visibilidad, versiones retenidas, aborts
Índice / EXPLAIN	¿Cómo encuentra filas sin leer todo?	plan, estimaciones, filas, buffers, tiempos
Compaction	¿Dónde se paga la escritura rápida?	bytes reescritos, L0, pending bytes, I/O
Stall	¿Cómo evita el motor una cola infinita?	tiempo en stall, p99, backlog y tasa sostenible

Las analogías sirven para comenzar. Una respuesta universitaria debe terminar especificando garantía, falla, mecanismo, costo y evidencia.

6 La jerarquía física y la frontera de durabilidad

6.1 Una llamada a `write()` no es necesariamente durabilidad

Entre una aplicación y el medio persistente existen varias capas. Una representación simplificada es:



Una llamada a `write()` puede terminar cuando el kernel aceptó los bytes en su cache, aunque todavía no hayan llegado al medio persistente. En caso de pérdida de energía, esos bytes pueden desaparecer. Por eso hay que distinguir:

- **aceptado:** alguna capa recibió la operación;
- **escrito:** los bytes salieron de ciertos buffers, pero no necesariamente son permanentes;
- **flushed / durable:** el sistema considera que los bytes cruzaron la frontera de persistencia definida;
- **aplicado:** la estructura principal refleja el cambio;
- **replicado:** otra copia recibió el cambio, con una semántica que debe especificarse.

Estas palabras no son intercambiables. Una arquitectura seria debe definirlas operacionalmente.

6.2 La frontera depende del modelo de falla

No existe una noción de durabilidad independiente del modelo de falla. Un flush local puede tolerar un crash de proceso y una pérdida de energía correctamente manejada por el dispositivo, pero no la destrucción física del disco. Una mayoría durable puede tolerar la pérdida de algunos nodos, pero introduce coordinación y puede quedar indisponible durante una partición.

En consecuencia, una garantía de durabilidad debería acompañarse de preguntas como:

- ¿frente a qué fallas se promete supervivencia?;
- ¿cuánta pérdida de datos reciente es aceptable, es decir, cuál es el RPO?;
- ¿cuánto tiempo puede tardar la recuperación, es decir, cuál es el RTO?;
- ¿qué hardware, configuración y protocolo sostienen esa promesa?;
- ¿qué evidencia permite auditar que la frontera se cruzó realmente?

6.3 Insert LSN, write LSN y flush LSN

PostgreSQL permite observar diferentes posiciones del WAL. Conceptualmente:

- *insert position*: final lógico de lo insertado en buffers internos;
- *write position*: datos escritos desde buffers del servidor hacia el sistema operativo;
- *flush position*: último punto conocido como durable.

Esta distinción ofrece un ejemplo concreto de por qué “generado”, “escrito” y “durable” no son sinónimos.

7 WAL: construir durabilidad con un orden físico

7.1 Problema que resuelve

Modificar cada página de datos y forzarla a disco antes de responder sería costoso. Además, una transacción puede afectar varias páginas: si el sistema cae después de escribir algunas pero no otras, quedaría una mezcla incoherente.

WAL resuelve este problema imponiendo un orden seguro entre una descripción compacta del cambio y las páginas modificadas.

7.2 La regla WAL

La regla tiene dos partes:

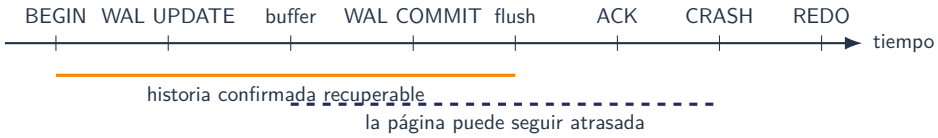
1. el registro que permite rehacer una modificación debe llegar al WAL durable antes de que la página modificada pueda llegar al almacenamiento principal;
2. el registro de commit debe ser durable antes de confirmar éxito al cliente.

Esto permite una política *no-force*: las páginas de datos no tienen que escribirse en cada commit. El log secuencial se persiste en el camino crítico; las páginas pueden drenarse después en un orden más eficiente.

WAL no elimina el trabajo de escribir páginas. Cambia el orden del riesgo: primero hace durable una historia recuperable y después materializa esa historia en las estructuras principales.

7.3 Timeline completo del caso

Una historia simplificada para la compra es:



Los eventos importantes son:

1. la transacción comienza;
2. se registra la información que permite rehacer la modificación;
3. la página en memoria cambia;
4. se agrega el registro de commit;
5. el WAL se fuerza hasta el commit;
6. recién entonces se confirma al cliente;
7. ocurre el crash antes de escribir la página;
8. recovery vuelve a aplicar el cambio confirmado.

7.4 Tres puntos de crash

Crash antes del commit durable. Puede existir trabajo en memoria e incluso registros de actualización en WAL, pero no hay evidencia durable de una decisión de commit. Recovery no debe exponer esa transacción como confirmada. El tratamiento exacto depende del motor: puede ignorar versiones no confirmadas, ejecutar UNDO o seguir otro protocolo equivalente.

Crash después del commit durable y antes de la página. La transacción fue confirmada correctamente. Recovery debe hacer REDO de los cambios que todavía no están reflejados en las páginas.

Crash después de página y commit. La página puede ya contener el valor. Recovery debe reconocer que el cambio está aplicado o volver a aplicarlo de manera segura. De ahí surge la necesidad de que REDO sea repetible.

7.5 REDO, UNDO y políticas de buffer

Dos decisiones clásicas ayudan a entender recovery:

- **STEAL / NO-STEAL:** si una página con cambios no confirmados puede llegar al disco antes del commit;
- **FORCE / NO-FORCE:** si todas las páginas modificadas deben forzarse al commit.

Una combinación frecuente es STEAL + NO-FORCE. Permite usar buffers con flexibilidad y evita forzar todas las páginas en cada commit, pero exige información suficiente para UNDO y REDO o

mecanismos equivalentes. ARIES es la referencia clásica para este estilo de recovery; PostgreSQL implementa decisiones propias, combinando WAL, MVCC y recovery de avance.

7.6 Por qué REDO debe ser repetible

Recovery puede sufrir otra caída y volver a comenzar. Si un registro dice "sumar 1500" y se aplica sin saber si ya ocurrió, una repetición podría duplicar el efecto. Los motores evitan esto mediante información de página, LSN y reglas de aplicación que permiten determinar si un registro ya está reflejado.

La intuición correcta es que REDO lleva la página al estado descrito por la historia confirmada; no repite ciegamente una operación de negocio.

7.7 Group commit

Forzar almacenamiento estable por cada transacción puede limitar throughput. Group commit agrupa varios commits y paga una sincronización física compartida. La garantía no se debilita si cada transacción recibe ACK solo después de que el grupo cruzó la frontera durable.

El trade-off es:

- mayor throughput y mejor amortización de I/O;
- espera compartida y cambios en la distribución de latencia;
- posible interacción con batching, colas y p99 bajo carga.

7.8 Commit asincrónico

Algunos motores permiten responder antes de que el commit sea flushed. Esto mejora latencia, pero crea una ventana en la que una transacción confirmada al cliente puede perderse ante una caída. No es "la misma durabilidad más rápida": es una garantía distinta que debe declararse.

8 Checkpoints y tiempo de recuperación

Si recovery tuviera que reproducir todo el WAL desde el origen, el tiempo de reinicio crecería sin límite. Un checkpoint establece un punto de referencia a partir del cual basta procesar una porción más reciente de la historia.

Conceptualmente, un checkpoint coordina:

- el estado de páginas modificadas;
- un punto conocido del WAL;
- retención del log necesario;
- información para que recovery sepa dónde comenzar.

El intervalo de checkpoint redistribuye costo:

- **muy frecuente:** menor WAL a reproducir, pero más presión de escritura y riesgo de picos;
- **muy espaciado:** menos trabajo periódico, pero más WAL retenido y mayor RTO.

La elección correcta no es "más checkpoints". Es una configuración compatible con RTO, capacidad de I/O y SLO de latencia normal.

9 Caso real: PostgreSQL y roll-forward recovery

La documentación oficial de PostgreSQL describe el principio de WAL: los cambios de archivos de datos solo deben escribirse después de que los registros que los describen hayan sido flushed a almacenamiento permanente. Si ocurre un crash, el servidor puede restaurar consistencia reproduciendo registros desde el último checkpoint.

Tres consecuencias didácticas son importantes:

1. un commit correcto no implica que todas las páginas modificadas ya estén en su ubicación final;
2. el log secuencial permite sacar I/O aleatorio del camino crítico;
3. la promesa depende también del dispositivo: si el hardware informa falsamente que un flush terminó, la regla puede quedar subvertida.

PostgreSQL también expone funciones para diferenciar la posición insertada, escrita y flushed del WAL, lo cual vuelve observable la frontera conceptual discutida antes.

10 MVCC: conservar historia para reducir esperas

10.1 Problema que resuelve

Supongamos que un reporte de ventas tarda 40 segundos y, al mismo tiempo, el sistema registra nuevas compras. Sin múltiples versiones, hay dos alternativas poco satisfactorias:

- bloquear escritores durante toda la lectura;
- permitir que el reporte mezcle datos de momentos diferentes.

MVCC conserva varias versiones y permite que la lectura observe un snapshot coherente mientras el presente continúa avanzando.

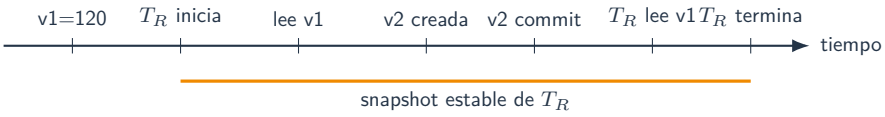
10.2 Modelo simplificado de versión

Una versión puede representarse conceptualmente como:

$$(x,\ valor,\ t_{inicio},\ t_{fin})$$

La versión es visible para un snapshot s si el cambio que la creó es visible en s y el cambio que la reemplazó todavía no lo era. En motores reales, estas reglas se implementan con identificadores de transacción, estados de commit y metadatos de fila o página.

10.3 Timeline de visibilidad



Aunque v2 se confirma durante la ejecución, T_R puede seguir viendo v1. Una transacción nueva, en cambio, podrá ver v2. Esto reduce bloqueo entre lectores y escritores.

10.4 MVCC no elimina el costo de concurrencia

Conservar historia tiene costos:

- más versiones ocupan espacio;
- índices pueden contener referencias a versiones que ya no son visibles;
- las páginas se vuelven menos densas;
- aumenta I/O y se degrada cache locality;
- se necesita garbage collection o vacuum;
- snapshots largos retrasan la reclamación.

Por eso una frase útil es:

> MVCC cambia espera por administración de historia; la historia también cuesta.

10.5 Caso real: PostgreSQL y VACUUM

En PostgreSQL, un `UPDATE` crea una nueva versión de fila. La versión anterior no puede eliminarse mientras una transacción activa aún pueda verla. Cuando ya no es visible para ningún snapshot relevante, `VACUUM` permite que el espacio vuelva a reutilizarse.

Una transacción muy larga puede mantener un horizonte antiguo y provocar:

- acumulación de tuplas muertas;
- bloat de tablas e índices;
- más páginas leídas por consulta;
- mayor trabajo de vacuum;
- presión sobre almacenamiento y cache.

El problema no es solamente de espacio. Una tabla inflada cambia el plan y la latencia de consultas.

10.6 MVCC y niveles de aislamiento

MVCC no es un nivel de aislamiento único. Un motor puede usar versiones para implementar Read Committed, Repeatable Read, Snapshot Isolation o Serializable. Las diferencias dependen de cuándo se toma el snapshot y qué conflictos o dependencias se validan.

Snapshot Isolation suele ofrecer lecturas coherentes y evitar conflictos write-write directos, pero puede admitir write skew.

10.7 Write skew

Supongamos dos filas A y B con el invariante "al menos una debe permanecer activa".

- T_1 lee A=1, B=1 y escribe A=0.
- T_2 lee A=1, B=1 y escribe B=0.

Cada transacción ve un snapshot válido y escribe una fila distinta. Si ambas confirman, el estado final A=0, B=0 viola el invariante. No hubo conflicto write-write sobre la misma fila.

Posibles respuestas incluyen:

- aislamiento serializable;
- lock explícito sobre un recurso común;
- reformular el invariante para crear una contención detectable;
- validación de dependencias al commit;
- operación atómica especializada.

La lección es que un snapshot coherente no equivale automáticamente a una historia serializable.

11 Índices y `EXPLAIN`: medir antes de optimizar

11.1 Qué es `EXPLAIN` y qué no es

`EXPLAIN` no "optimiza" la consulta ni garantiza que el plan sea bueno. Expone el plan que el optimizador eligió a partir de estadísticas y un modelo de costo. Debe leerse desde los nodos hoja hacia la raíz: primero cómo se obtienen filas, luego cómo se filtran, combinan, ordenan o limitan.

`EXPLAIN ANALYZE` ejecuta realmente la consulta. Por eso debe utilizarse con cuidado sobre operaciones que modifican datos o sobre cargas productivas sensibles. Los tiempos de una ejecución aislada no constituyen una promesa universal; sí permiten contrastar forma del plan, cardinalidad estimada y observada, buffers y trabajo descartado.

> Pregunta de lectura: no pregunte primero "¿cuántos milisegundos tardó?". Pregunte "¿qué trabajo hizo el motor, cuántas filas produjo cada nodo y qué evidencia muestra que ese trabajo era necesario?".

11.2 Por qué un B-tree mantiene poca altura

Un B-tree almacena muchos separadores y punteros por página. Su alto fan-out produce árboles poco profundos. Si una página interna puede contener cientos de punteros, incluso millones de claves requieren pocas lecturas de nivel.

El B-tree es un índice generalista adecuado para:

- igualdad;
- rangos;
- ordenamiento;
- búsqueda del siguiente o anterior;
- prefijos compatibles con la estructura del índice.

11.3 El costo de mantener índices

Cada índice adicional puede aumentar:

- trabajo de `INSERT`, `UPDATE` y `DELETE`;
- volumen de WAL;
- número de páginas modificadas;

- espacio y cache pressure;
- splits;
- trabajo de vacuum o reindexación.

Por lo tanto, "crear un índice" no es una optimización gratuita. Es intercambiar costo de lectura por costo de escritura, espacio y mantenimiento.

11.4 Consulta del caso conductor

La app necesita mostrar las últimas 20 compras de una estudiante:

```
SELECT id, total, created_at
FROM campus_purchases
WHERE student_id = 42
ORDER BY created_at DESC
LIMIT 20;
```

Un índice solo sobre `student_id` puede localizar las filas de la estudiante, pero quizá deba ordenar después. Un índice compuesto:

```
CREATE INDEX idx_purchases_student_created
ON campus_purchases (student_id, created_at DESC);
```

puede satisfacer filtro y orden, permitiendo cortar tras 20 entradas.

11.5 Salida de `EXPLAIN` sin índice

Una salida didáctica simplificada puede mostrar:

```
Limit  (actual time=86.411..86.417 rows=20)
  -> Sort  (top-N heapsort)
       Sort Key: created_at DESC
       -> Seq Scan on campus_purchases
            Filter: (student_id = 42)
            Rows Removed by Filter: 998752
Execution Time: 86.450 ms
```

La forma del trabajo es:

1. recorrer casi toda la tabla;
2. descartar filas de otros estudiantes;
3. ordenar las coincidencias;
4. conservar 20.

11.6 Salida con índice compuesto

```
Limit  (actual time=0.035..0.041 rows=20)
  -> Index Scan using idx_purchases_student_created
       on campus_purchases
       Index Cond: (student_id = 42)
Execution Time: 0.071 ms
```

El índice entrega primero las entradas de la clave buscada y en el orden requerido. El `LIMIT` puede cortar temprano.

Los tiempos anteriores son ilustrativos y no deben tratarse como benchmark reproducible. Lo importante es aprender a leer la forma del plan.

11.7 Qué observar en `EXPLAIN (ANALYZE, BUFFERS)`

- tipo de scan;
- costo estimado;
- filas estimadas frente a reales;
- filas descartadas por filtros;
- operaciones de sort;
- loops;
- buffers hit/read;
- tiempo total y variabilidad entre ejecuciones.

Una discrepancia grande entre filas estimadas y reales puede indicar estadísticas desactualizadas, correlación de columnas o distribución que el planner no modeló bien.

11.8 Selectividad y cuándo un seq scan es correcto

Si una consulta devuelve gran parte de la tabla, usar el índice puede implicar demasiados accesos dispersos al heap. El planner puede preferir un scan secuencial incluso cuando existe un índice. Eso no significa automáticamente que el índice "no funciona"; significa que para esa consulta y esos datos otra estrategia parece más barata.

11.9 Índices compuestos y orden de columnas

El orden de columnas define qué prefijos son útiles. Para una consulta que fija `student_id` y recorre fechas, resulta natural un índice con `student_id` como primera columna y `created_at` como segunda. Invertir ese orden produce una estructura físicamente distinta y no ofrece el mismo acceso.

La elección debería partir de consultas reales y de su distribución, no solo del esquema lógico.

11.10 Covering e índices parciales

Un índice puede incluir columnas para permitir un index-only scan en determinadas condiciones, aunque en PostgreSQL la visibilidad de la fila también influye. Los índices parciales permiten indexar solo un subconjunto, por ejemplo compras pendientes, reduciendo tamaño y costo cuando ese subconjunto es pequeño y consultado con frecuencia.

Estas técnicas son útiles, pero aumentan especificidad y deben justificarse con carga real.

12 LSM y compaction: throughput ahora, deuda después

12.1 Motivación

Una Log-Structured Merge-Tree transforma muchas escrituras pequeñas y aleatorias en appends y merges secuenciales. Es atractiva para cargas de ingestión intensa, stores clave-valor, series temporales y telemetría.

La arquitectura típica contiene:

- WAL para durabilidad;
- memtable ordenada en RAM;
- immutable memtable pendiente de flush;
- SSTables inmutables en disco;
- niveles o tiers;
- compaction en background;
- metadata o manifest para describir los archivos activos.

12.2 Ruta de escritura

Una escritura sigue, conceptualmente, estos pasos:

1. append al WAL;
2. inserción en la memtable;
3. respuesta según la política de durabilidad;
4. congelamiento de la memtable cuando alcanza un umbral;
5. flush secuencial a una SSTable L0;
6. compaction hacia niveles inferiores.

El camino crítico es corto, pero el sistema adquiere una deuda de organización que debe pagar después.

12.3 SSTables

Una SSTable es un archivo inmutable con claves ordenadas. La inmutabilidad simplifica escritura secuencial y concurrencia. Sin embargo, una clave puede aparecer en varias SSTables con diferentes versiones, y un tombstone puede representar una eliminación que todavía no puede descartarse.

12.4 Ruta de lectura

Una búsqueda puede consultar:

1. memtable activa;
2. immutable memtables;
3. archivos L0, que pueden solaparse;
4. niveles compactados.

Para reducir trabajo se usan:

- Bloom filters;

- índices por archivo y por bloque;
- block cache;
- metadata de rangos;
- compaction que controla solapamiento.

12.5 Bloom filters

Un Bloom filter puede afirmar con certeza que una clave no está en un archivo. Si responde "quizá", el archivo debe consultarse. Por ello:

- falsos positivos generan I/O adicional;
- falsos negativos no deben ocurrir;
- más bits por clave reducen falsos positivos, pero consumen memoria y metadata.

Son especialmente útiles para point lookups de claves ausentes; ayudan menos en grandes range scans.

12.6 Tombstones

Como las SSTables son inmutables, una eliminación se registra como tombstone. Ese tombstone debe propagarse durante compaction hasta que sea seguro descartar tanto la marca como las versiones antiguas. Eliminarlo demasiado pronto podría hacer reaparecer una versión vieja almacenada en otro nivel.

13 Compaction y stalls: pagar y regular la deuda diferida

13.1 De la intuición a la operación física

La analogía de "ordenar pilas" es útil porque deja visible el costo diferido. La aplicación escribe una versión una vez; el motor puede leerla y reescribirla varias veces a medida que la mueve entre niveles. Ese trabajo compite con lecturas y nuevas escrituras. La capacidad sostenida del sistema depende tanto del camino de ingreso como del trabajo de mantenimiento.

13.2 Por qué existe

Compaction fusiona runs ordenados, resuelve versiones, propaga tombstones y reorganiza rangos. Sin compaction, crecerían el número de archivos, read amplification y space amplification.

13.3 Write amplification

$$WA = \frac{\text{bytes físicos escritos}}{\text{bytes lógicos ingresados}}$$

Un dato puede escribirse en WAL, SSTable L0 y varias compactaciones. Índices, replicación y backups pueden agregar más escritura fuera del motor.

13.4 Read amplification

Read amplification mide cuántas estructuras, archivos o bloques deben consultarse para responder. En una LSM aumenta cuando existen muchos runs o solapamientos. Bloom filters y cache la reducen, pero no la eliminan.

13.5 Space amplification

$$SA = \frac{\text{espacio físico ocupado}}{\text{estado lógico vivo}}$$

Versiones, tombstones, archivos solapados y espacio temporal de compaction contribuyen.

13.6 Leveled compaction

En leveled compaction, los niveles mayores buscan mantener rangos poco solapados. Esto favorece lectura y control de espacio, pero puede reescribir datos varias veces al moverlos entre niveles.

13.7 Tiered o universal compaction

Conserva varios runs y los fusiona por grupos. Puede reducir parte de la reescritura temprana, pero aumenta el número de runs consultados y el espacio temporal. Es una redistribución de costo, no una mejora absoluta.

13.8 Write stalls

Si la tasa de ingreso supera sostenidamente la capacidad de flush y compaction, se acumulan immutable memtables, archivos L0 y bytes pendientes. El motor puede desacelerar o detener escrituras para evitar degradación sin límite.

RocksDB documenta write stalls como una forma de backpressure. Sin ellos, el sistema podría aumentar space amplification hasta quedarse sin disco y read amplification hasta volver inutilizables las consultas.

13.9 Métricas de compaction

Las métricas más útiles incluyen:

- bytes leídos y escritos por compaction;
- bytes pendientes de compaction;
- cantidad de archivos L0;
- número y duración de stalls;
- tiempo total en stall;
- write amplification;
- latencia p95/p99 de escritura y lectura;
- utilización del dispositivo;
- throughput de flush/compaction frente a ingreso.

Una salida donde ingresan 90 MB/s, compaction procesa 60 MB/s y el backlog crece no describe un pico inocente si la diferencia persiste. Describe una cola inestable. El throughput aceptado en el frente no es sostenible.

13.10 Leer un dashboard de compaction como una cadena causal

Considere una ventana de cinco minutos con las siguientes señales ilustrativas:

Señal	Valor	Pregunta que habilita
User bytes	120 GB	¿cuánto dato lógico ingresó?
Compaction write	780 GB	¿cuánto trabajo físico adicional se pagó?
Write amplification	$6.5\times$	¿es compatible con SSD, costo y vida útil?
Pending compaction bytes	18 GB	¿la deuda crece o se drena?
L0 files	28 de 36	¿qué tan cerca está el umbral de stop?
Stall time	3.4 s/min	¿cuánto backpressure ya siente la aplicación?

Ninguna señal aislada explica el sistema. La interpretación correcta relaciona tasas y tendencias:

1. si ingestión supera compaction, el backlog crece;
2. al crecer L0 y solapamiento, aumenta trabajo de lectura;
3. más compaction compite por I/O y puede elevar p99;
4. el motor aplica slowdown o stop para recuperar estabilidad;
5. si la diferencia de capacidad persiste, el tuning solo redistribuye el problema: hay que reducir ingreso, aumentar capacidad o cambiar la arquitectura.

14 Caso contrastante: telemetría del campus

Sensores de aulas, energía y ocupación generan decenas de miles de eventos por segundo. La carga presenta:

- escrituras append;
- claves por dispositivo y tiempo;
- consultas por rango temporal;
- retención y expiración;
- vistas analíticas que toleran cierta frescura eventual.

Una LSM-tree puede ser una hipótesis razonable, pero todavía deben definirse:

- política de WAL y pérdida máxima aceptable;
- tamaño de memtable;
- estrategia de compaction;
- particionamiento temporal;
- Bloom filters;
- TTL y manejo de tombstones;
- métricas de backlog y stalls;
- recuperación y replicación.

Decir "usaremos RocksDB porque escribe rápido" no constituye una defensa. La defensa necesita un modelo de carga y un experimento.

15 B-tree y LSM: comparación razonada

Criterio	B-tree / páginas	LSM-tree / runs
Write path	Actualización de páginas e índices; más I/O aleatorio	WAL, memtable, flush y compaction diferida
Point lookup	Pocas páginas, comportamiento estable si la cache está sana	Puede consultar varios runs; Bloom y cache ayudan
Range scan	Natural por orden del árbol	Eficiente si hay pocos runs y buena compaction
Mantenimiento	Splits, vacuum, bloat y reindex	Tombstones, compaction, stalls y amplificaciones
Carga típica	OLTP equilibrado o read-heavy	Ingestión sostenida y write-heavy
Pregunta crítica	¿Qué consulta justifica cada índice?	¿Puede compaction sostener la tasa de ingreso?

La comparación "B-tree para lectura, LSM para escritura" es una orientación inicial, no una ley. Cache, hardware, tamaño de valores, compresión, patrón de rangos, distribución de claves y configuración pueden cambiar el resultado.

16 Dos decisiones de arquitectura

16.1 Compra en campus

Características:

- transacciones cortas;
- consulta de compras por estudiante y fecha;
- actualización de inventario;
- necesidad de idempotencia y durabilidad;
- latencia p99 predecible.

Una hipótesis inicial razonable es PostgreSQL u otro motor transaccional con WAL, MVCC y B-tree. La decisión debe validarse con:

- p95/p99 de checkout;
- tasa de commits;
- latencia de flush;
- tiempo de recovery;
- bloat y vacuum;
- uso y costo de índices;
- comportamiento bajo concurrencia e invariantes.

16.2 Telemetría del campus

Características:

- ingestión masiva;
- append por dispositivo y timestamp;
- rangos temporales;
- retención;
- tolerancia a frescura eventual en vistas.

Una hipótesis inicial es una LSM con particionamiento temporal. Las métricas incluyen:

- throughput sostenible, no solo pico;
- backlog de compaction;
- write/read/space amplification;
- p99 de escritura durante compaction;
- latencia de rangos;
- espacio temporal;
- stalls;
- recovery y pérdida máxima.

17 Método reusable para decidir storage

Una decisión madura puede organizarse en seis pasos:

1. **Describir la carga.** Ratio de lectura/escritura, tamaño de claves y valores, skew, crecimiento, rangos, concurrencia y retención.
2. **Declarar garantías.** Durabilidad, RPO, RTO, aislamiento, consistencia y frescura.
3. **Elegir una hipótesis de mecanismo.** WAL, MVCC, B-tree, LSM, particionamiento y replicación.
4. **Identificar el costo desplazado.** Flush, vacuum, compaction, cache, espacio, coordinación o recovery.
5. **Definir métricas.** Percentiles de latencia y throughput sostenible. Agregar amplificaciones, backlog, hit rate y objetivos RPO/RTO.
6. **Diseñar un experimento que pueda refutar la elección.** No solo demostrar el caso feliz.

> Una arquitectura queda mejor defendida cuando incluye la condición bajo la cual el equipo estaría dispuesto a cambiarla.

18 Demostración reproducible de `EXPLAIN`

El paquete de esta clase incluye un script SQL que:

1. crea una tabla de compras del campus;

2. genera un conjunto sintético con `generate_series`;
3. ejecuta `EXPLAIN (ANALYZE, BUFFERS)` sin índice;
4. crea un índice compuesto;
5. repite la consulta;
6. permite discutir qué cambió en el plan y qué costo se agregó a escrituras.

Los tiempos concretos dependen de hardware, cache y tamaño del dataset. La evidencia principal no es una cifra aislada, sino la diferencia entre `Seq Scan + Sort` e `Index Scan` compatible con filtro, orden y límite.

19 Aplicación del marco del curso

Para la telemetría del campus:

$$D = (P, F, G, S, T)$$

- P: ingerir y consultar eventos a alta tasa;
- F: crash, pérdida de dispositivo, picos, compaction atrasada, saturación y partición;
- G: pérdida máxima acotada, recuperación dentro del objetivo y rangos consultables;
- S: WAL, LSM, replicación, compaction, retención y backpressure;
- T: write amplification, latencia de compaction, más complejidad y consistencia eventual en vistas.

Para la compra en campus:

- P: confirmar compras exactamente una vez a nivel lógico;
- F: timeout, retry, crash, pérdida de energía y concurrencia;
- G: durabilidad, idempotencia, inventario coherente y p99 acotado;
- S: transacción, WAL durable, MVCC, índice compuesto y recuperación;
- T: fsync, mantenimiento de índices, vacuum y coordinación.

20 Precisiones técnicas indispensables

Esta sección reúne distinciones que suelen provocar respuestas aparentemente correctas pero conceptualmente incompletas.

20.1 WAL no produce exactamente una vez por sí solo

WAL permite reconstruir una historia confirmada después de un crash. No identifica automáticamente que dos requests sean la misma operación lógica. Para una compra reintentada hacen falta, además, una clave idempotente, una restricción de unicidad y la persistencia atómica del efecto y del resultado asociado a esa clave.

20.2 La regla WAL contiene dos órdenes diferentes

Primero, el registro que explica una modificación debe ser durable antes de que la página modificada alcance almacenamiento estable. Segundo, el registro de commit debe ser durable antes del ACK al cliente. El primer orden protege recovery; el segundo protege la promesa externa de durabilidad.

20.3 Checkpoint, backup y réplica responden preguntas distintas

Un checkpoint acota cuánto log debe releerse. Un backup permite recuperar frente a pérdida o corrupción más amplia. Una réplica mejora disponibilidad y puede reducir pérdida, pero también puede reproducir una corrupción lógica. Ninguno reemplaza automáticamente a los otros.

20.4 MVCC es un mecanismo; el aislamiento es el contrato

Conservar versiones reduce bloqueo entre lectores y escritores. Las reglas de snapshot, detección de conflictos y validación de commit determinan si el contrato resultante es Read Committed, Snapshot Isolation o Serializable. Preguntar solamente "¿usa MVCC?" no alcanza.

20.5 El planner compara costos; no obedece la existencia de un índice

Un índice puede ser ignorado cuando la consulta devuelve una fracción grande de la tabla, las estadísticas son incorrectas, el orden del índice no coincide con el predicado o el acceso aleatorio cuesta más que un scan secuencial. La evidencia relevante es el plan y la distribución real de los datos.

20.6 Compaction es capacidad de mantenimiento

Una LSM es estable cuando flush y compaction pueden absorber, en promedio y bajo picos razonables, el trabajo que la aplicación crea. Un stall ocasional puede proteger al motor; stalls frecuentes indican que la capacidad de mantenimiento, la política de niveles o el hardware no sostienen la carga.

21 Glosario operativo mínimo

Aceptado: una capa recibió la operación; no dice cuánto sobrevivirá.

Escrito: los bytes fueron entregados a alguna capa inferior; puede seguir siendo volátil.

Durable: la operación cruzó la frontera de persistencia declarada para el modelo de falla.

Committed: la transacción tomó una decisión final según el protocolo; debe aclararse si esa decisión es durable.

Aplicado: la estructura principal refleja el cambio; puede ocurrir después del commit durable.

Visible: una lectura puede observar el cambio según su snapshot y nivel de aislamiento.

Replicado: otra copia recibió el cambio; debe aclararse si fue memoria, disco o quorum.

Backlog: trabajo pendiente que el sistema todavía debe absorber.

Stall: regulación deliberada de admisión para evitar crecimiento ilimitado del backlog.

22 Errores conceptuales frecuentes

- confundir `write()` exitoso con durabilidad;

- suponer que replicación sustituye a persistencia;
- pensar que `COMMIT` implica páginas ya escritas;
- creer que WAL por sí solo implementa ACID completo;
- afirmar que MVCC elimina conflictos o garantiza serializabilidad;
- crear índices sin observar consultas, selectividad ni costo de escritura;
- tratar compaction como mantenimiento gratuito;
- comparar motores solo mediante throughput medio;
- ignorar p99, backlog y recuperación;
- elegir una tecnología por reputación sin formular una carga falsable.

23 Bibliografía guiada

- Martin Kleppmann, *Designing Data-Intensive Applications*, capítulos 3 y 7. Leer primero para construir el modelo conceptual de estructuras de almacenamiento, índices y transacciones.
- PostgreSQL, documentación oficial, capítulo de Reliability and the Write-Ahead Log. Concentrarse en la regla WAL, configuración y la advertencia sobre dispositivos que informan flush de manera incorrecta.
- PostgreSQL, documentación oficial de `EXPLAIN` e índices. Practicar la lectura de scans, sorts, estimaciones, filas reales y buffers.
- PostgreSQL, documentación de MVCC y routine vacuuming. Relacionar versiones visibles, snapshots largos y bloat.
- RocksDB Wiki, *Compaction Stats and DB Status*, *Write Stalls* y *Tuning Guide*. Observar cómo backlog, L0 y stalls convierten compaction en un problema operacional.
- Patrick O'Neil et al., "The Log-Structured Merge-Tree", *Acta Informatica*, 1996. Referencia fundacional de LSM-trees.
- Mohan et al., "ARIES: A Transaction Recovery Method Supporting Fine-Granularity Locking and Partial Rollbacks Using Write-Ahead Logging", ACM TODS, 1992. Lectura avanzada sobre recovery.

Ruta sugerida: DDIA para el mapa; PostgreSQL para una implementación transaccional observable; RocksDB para compaction y backpressure; artículos fundacionales para profundización.

24 Conexión con el Trabajo Final Integrador

La estrategia de datos del Trabajo Final no debe expresarse como "usaremos PostgreSQL", "usaremos Cassandra" o "usaremos RocksDB". Debe incluir:

- patrón de lectura y escritura;
- volumen, crecimiento y skew;
- dato crítico e invariante;
- frontera de durabilidad;

- RPO y RTO;
- nivel de aislamiento;
- estrategia de índices o particionamiento;
- recovery, backups y replicación;
- costos de vacuum o compaction;
- métricas y experimento de validación.

25 Autoevaluación

Responda sin mirar el texto y luego vuelva a cada sección para justificar con una regla o una métrica.

1. ¿Por qué una página de datos puede estar atrasada después de un commit correcto?
2. ¿Qué diferencia existe entre insert, write y flush del WAL?
3. ¿Qué falla tolera un WAL local que no tolera una réplica en RAM, y viceversa?
4. ¿Por qué REDO debe poder repetirse?
5. ¿Qué cambia group commit y qué no cambia?
6. ¿Cómo afecta el intervalo de checkpoint al RTO y a la latencia normal?
7. ¿Qué snapshot ve una transacción larga en MVCC?
8. ¿Por qué una transacción larga puede producir bloat?
9. ¿Por qué MVCC no implica serializabilidad?
10. ¿Qué trabajo evita el índice $(student_id, created_at)$ en el caso estudiado?
11. ¿Por qué un seq scan puede ser correcto aunque exista un índice?
12. ¿Qué costo introduce cada índice adicional?
13. ¿Cómo se escribe un dato en una LSM?
14. ¿Qué problema resuelven Bloom filters y qué error admiten?
15. ¿Por qué no puede eliminarse inmediatamente un tombstone?
16. ¿Qué significa write amplification?
17. ¿Cómo se detecta que compaction no sostiene el ritmo de ingreso?
18. ¿Por qué un write stall puede proteger al sistema?
19. ¿Qué mediría para defender B-tree frente a LSM en una carga concreta?
20. ¿Qué evidencia podría hacerle cambiar la arquitectura elegida?

26 Síntesis final

WAL hace recuperable una historia confirmada; los checkpoints acotan el trabajo de recovery; MVCC conserva historias concurrentes para reducir bloqueo; B-trees organizan acceso por páginas; LSM-trees convierten appends en trabajo diferido de compaction. Ningún mecanismo es gratuito.

El diseño correcto de almacenamiento conecta carga, falla, garantía, mecanismo, costo y evidencia. La pregunta central no es "¿qué motor es más rápido?", sino "¿qué promesa debe sobrevivir, qué trabajo se desplaza y cómo sabremos si la decisión sigue siendo válida?"